

Experiment No.	1
Experiment Title	Working with Python Packages – Numpy, Scipy, Scikit-Learn, Matplotlib ¹
Student Name	Sayed Afras S
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

The main aim of this experiment is to get comfortable with the basic Python packages that we will keep using throughout this lab – Numpy, Pandas, Scipy, Scikit-learn and Matplotlib. Instead of just reading about the functions, the idea here is to actually build a small reusable pipeline (EDA, preprocessing, feature selection, splitting and evaluation as separate functions) and run it on a real dataset, so that the workflow becomes clear before we jump into actual algorithms in the next experiments. Along with that, five different public datasets are studied to identify what kind of ML problem they represent (classification, regression, supervised/unsupervised etc.) and what feature selection technique and algorithm would suit each one.

2 Problem Statement

Given a raw, uncleaned dataset, the task is to explore it using standard Python data-science libraries, clean and preprocess it, select the most relevant features, split it into train/validation/test sets, fit an appropriate model and finally evaluate the model using suitable metrics. This experiment does not focus on any one specific ML algorithm – the goal is more about building intuition for the *end-to-end workflow* that almost every ML project follows, and writing the code in a reusable, modular way (functions that can be called again for any dataset in later experiments, not just this one).

As part of the theory portion of the experiment, five publicly available datasets were also studied and the type of ML task associated with each of them is summarised below.

Dataset – ML Task Identification

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset	Supervised – Multiclass Classification	ANOVA F-test (SelectKBest)	K-Nearest Neighbours / Decision Tree
Loan Amount Prediction	Supervised – Regression	Correlation analysis + SelectKBest (f_regression)	Linear Regression / Random Forest Regressor
Predicting Diabetes	Supervised – Binary Classification	Chi-square test / SelectKBest	Logistic Regression / SVM

Classification of Email Spam	Supervised – Binary Classification (Text)	Chi-square test on TF-IDF features	Naive Bayes (Multinomial)
Handwritten Character Recognition (MNIST)	Supervised – Multiclass Classification (Image)	PCA (dimensionality reduction, not exactly feature selection but serves same purpose here)	CNN / SVM

Note: Iris and MNIST are the most straight forward ones since they are already clean and labelled, so they are pure classification problems. Loan amount is a regression task because the target (amount) is continuous. Diabetes prediction and spam detection are both binary classification, the only difference is spam works on text data so TF-IDF vectors are used instead of numeric features directly.

3 Dataset Description

For the coding part of this experiment, instead of using Iris (which is a bit too clean and gets boring after the 100th time), the **Electric Car Data** dataset from Kaggle was used to actually run and test the reusable functions. This is a small dataset with details of different electric vehicles like acceleration, top speed, range, price etc, which makes it good for practicing regression along with the usual EDA/preprocessing steps.

Dataset Name	Electric Car Data (ElectricCarData_Clean.csv)
Dataset Source	Kaggle – https://www.kaggle.com/datasets/geoffnel/evs-one-electric-vehicle-dataset
Number of Samples	103
Number of Features	13 (excluding target)
Number of Classes	Not applicable – Regression target (PriceEuro)
Missing Values	None found for the numeric columns used, but FastCharge_KmH had a few non-numeric entries which were handled during preprocessing
Train-Test Split	70% Train, 10% Validation, 20% Test (using <code>train_test_split</code> twice)

4 Exploratory Data Analysis

A single reusable function `perform_eda()` was written which takes any dataframe and automatically prints the shape, head, tail, info, statistical summary, duplicate count and also generates a 6x2 grid of plots (missing values, correlation heatmap, histogram,

boxplot, scatter, category counts, pie chart, feature means, violin plot, density plot, outlier counts and a small summary box). This way the same function can just be called again for any dataset in the future experiments without re-writing all this code every time.

Listing 1: Reusable EDA function (trimmed for space, full version in notebook)

```

1 def perform_eda(df, filename="eda_summary.eps"):
2     print("DATASET SHAPE")
3     print(df.shape)
4     print("FIRST 5 ROWS")
5     print(df.head())
6     ...
7     num = df.select_dtypes(include=np.number)
8     cat = df.select_dtypes(exclude=np.number)
9     fig, axes = plt.subplots(6, 2, figsize=(40, 50))
10    axes = axes.flatten()
11    # Missing Values, Correlation Heatmap, Histogram,
12    # Boxplot, Scatter, Countplot, Pie Chart, Feature Means,
13    # Violin, Density, Outlier Count, Summary Table
14    ...
15    plt.savefig(filename, dpi=600, format="eps", bbox_inches="
        tight")
16    plt.show()

```

Running this on the raw dataset gave the following shape and summary:

Listing 2: Console output – EDA before preprocessing

```

1 DATASET SHAPE
2 (103, 14)
3 DATA TYPES
4 Brand                object
5 Model                object
6 AccelSec             float64
7 TopSpeed_KmH         int64
8 Range_Km             int64
9 Efficiency_WhKm       int64
10 FastCharge_KmH       object
11 RapidCharge          object
12 PowerTrain          object
13 PlugType            object
14 BodyStyle            object
15 Segment              object
16 Seats               int64
17 PriceEuro           int64
18 DUPLICATE ROWS
19 0

```

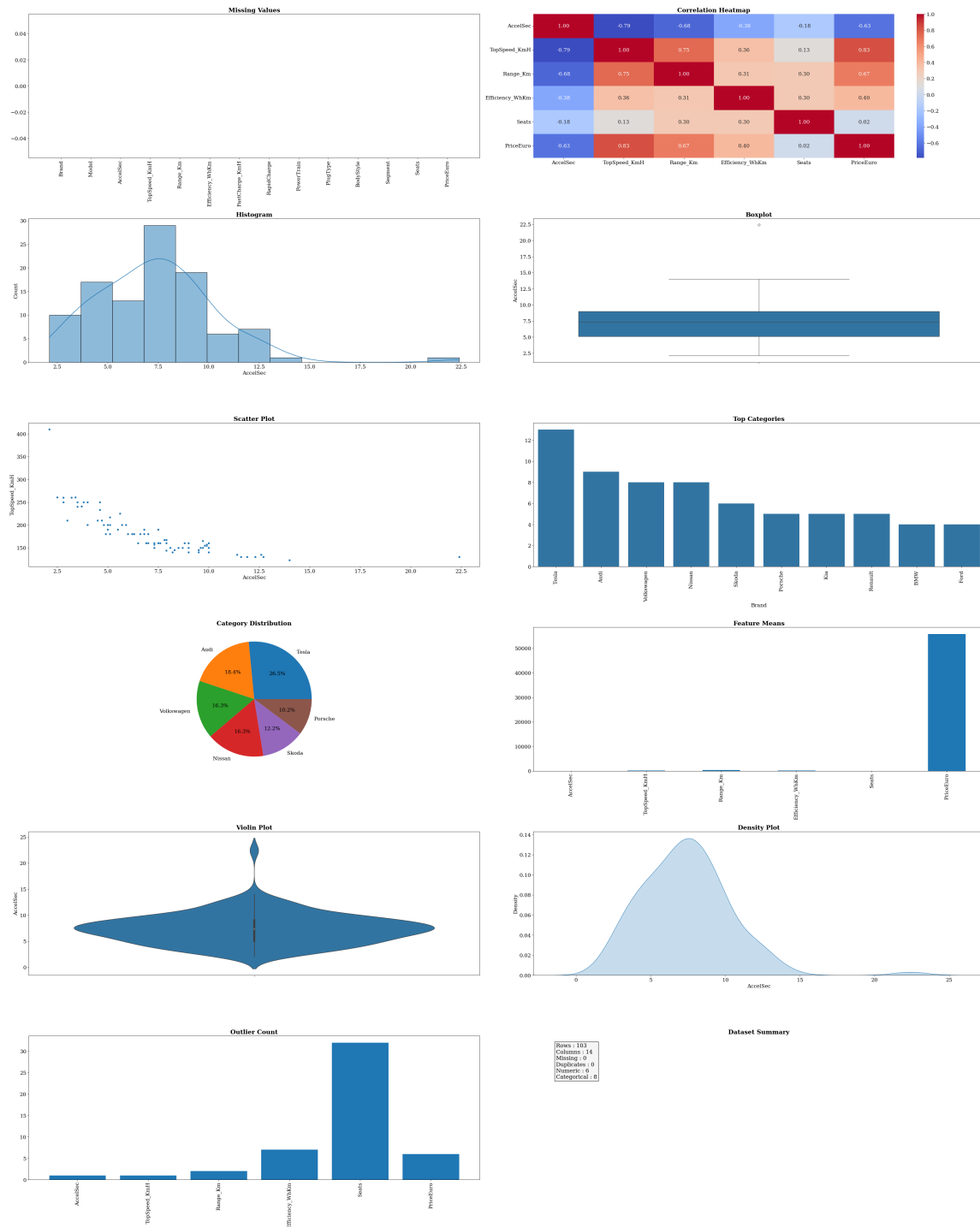


Figure 1: EDA summary grid for the Electric Car Data before preprocessing

Inference:

There are no duplicate rows in the dataset which is good, and also no completely missing cells in the numeric columns, so we do not have to worry too much about imputing large chunks of data. But the **FastCharge_KmH** column is stored as text/object type because some rows have a non numeric placeholder value in them, this had to be fixed before scaling. The correlation heatmap shows that **Range_Km** and **TopSpeed_KmH** have a decent positive correlation with the price, which kind of makes sense because faster and longer-range EVs are usually the expensive ones. The outlier plot also shows a couple of

cars (probably the premium brands like Tesla or Porsche models) sitting way outside the IQR range for price and top speed.

5 Data Preprocessing

The preprocessing step is handled by a single function `preprocess_data()` which takes a bunch of optional parameters (which columns to drop, how to fill missing numeric/categorical values, which encoding and scaling strategy to use, and how to treat outliers). Keeping it parameterised like this means it can be reused for classification tasks too later, we just change the arguments instead of writing new code.

Listing 3: Reusable preprocessing function

```

1 def preprocess_data(df, drop_columns=None, missing_num="mean",
2                     missing_cat="mode", encoding="label",
3                     scaling="standard", outliers="iqr"):
4     df = df.copy()
5     if drop_columns:
6         df.drop(columns=drop_columns, inplace=True, errors="
            ignore")
7
8     numeric_cols = df.select_dtypes(include=np.number).columns
9     categorical_cols = df.select_dtypes(exclude=np.number).
        columns
10
11     for col in numeric_cols:
12         if df[col].isnull().sum() > 0:
13             if missing_num == "mean":
14                 df[col].fillna(df[col].mean(), inplace=True)
15
16     for col in categorical_cols:
17         if df[col].isnull().sum() > 0:
18             df[col].fillna(df[col].mode()[0], inplace=True)
19
20     if outliers == "iqr":
21         for col in numeric_cols:
22             Q1, Q3 = df[col].quantile(0.25), df[col].quantile
                (0.75)
23             IQR = Q3 - Q1
24             df[col] = df[col].clip(Q1 - 1.5*IQR, Q3 + 1.5*IQR)
25
26     if encoding == "label":
27         le = LabelEncoder()
28         for col in categorical_cols:
29             df[col] = le.fit_transform(df[col].astype(str))
30
31     if scaling == "standard":
32         scaler = StandardScaler()
33         df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
34     return df

```

For this run the default options were used, so: missing numeric values were filled using mean, missing categorical values with mode, outliers were clipped using the IQR rule (not removed, just clipped to the upper/lower bound, so we don't lose any rows), categorical columns like Brand, PlugType, BodyStyle etc were Label Encoded, and finally all the numeric columns (including the encoded ones) were standard scaled. After running this the same `perform_eda()` function was called again just to double check everything looks fine.

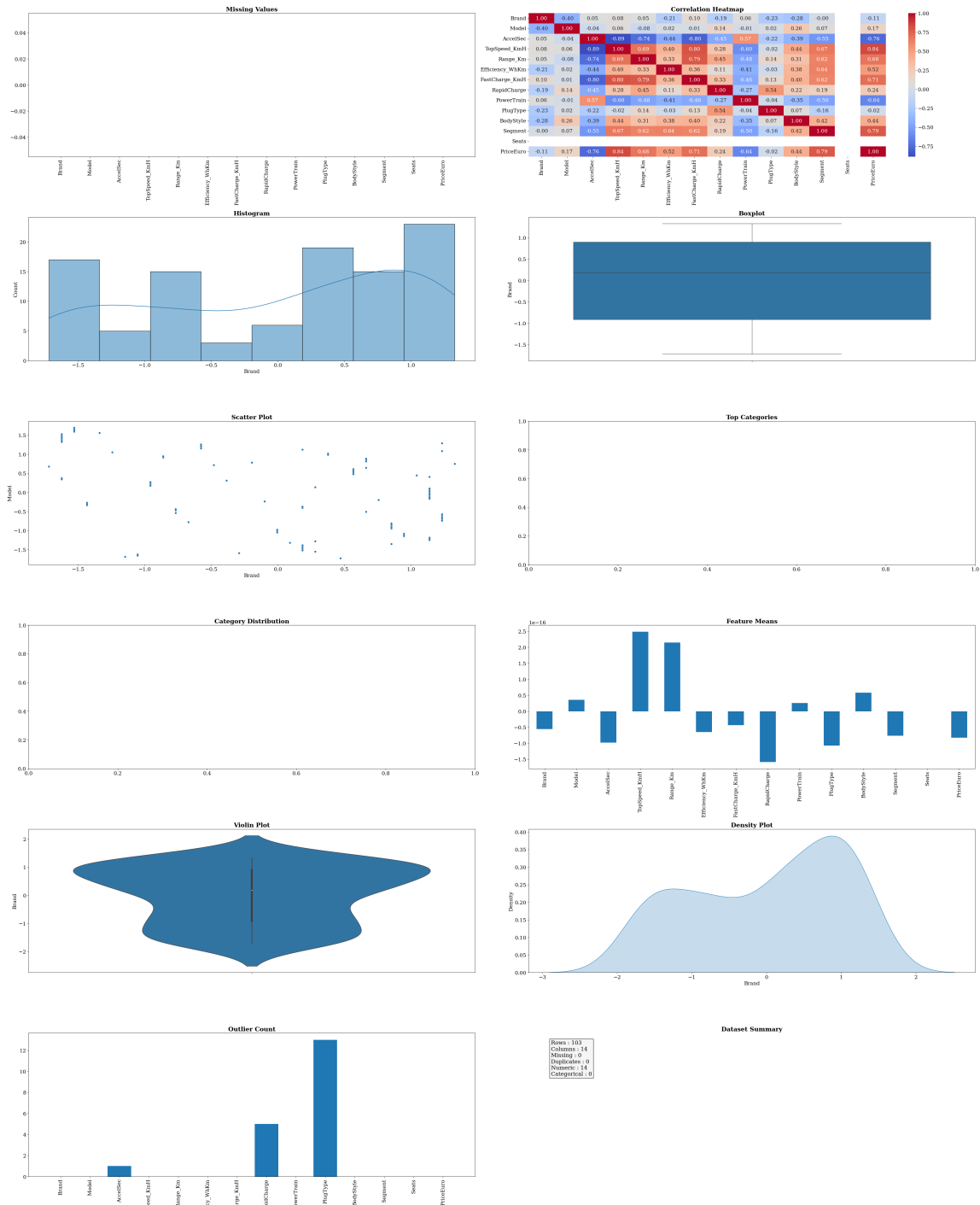


Figure 2: EDA summary grid for the Electric Car Data after preprocessing

Inference:

Once scaling is applied all the numeric features are now centered around 0 with unit variance, which is exactly what we want before feeding this into a distance/gradient based model like Linear Regression. The outlier count also drops a bit for a few columns since clipping pulled the extreme values back inside the IQR fence instead of leaving them as is. Looking at the histogram/density plot after scaling, most of the columns look roughly bell shaped now which is a good sign, although a couple of columns (like Seats) are still pretty skewed because they only take a small number of discrete values, scaling can't really fix that.

6 Machine Learning Algorithm

Since the target column here, `PriceEuro`, is continuous and has way more than 10 unique values, the pipeline automatically decided this is a **regression** problem and used **Linear Regression** from scikit-learn (this logic is written inside the `main()` function using a simple check `if y.nunique() <= 10` to switch between classification and regression).

- **Working principle:** Linear Regression tries to find a straight line (or hyperplane in higher dimensions) relationship between the input features and target by minimising the sum of squared errors between the predicted and actual values.
- **Mathematical formulation:**

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

where the weights w are found by minimising the cost function

$$J(w) = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

- **Advantages:** Simple, fast to train, easy to interpret the coefficients, works decently well when the relation between features and target is actually linear.
- **Limitations:** Assumes a linear relationship which is not always true, sensitive to outliers even after we clip them somewhat, and it does not handle multicollinearity between features very well (which is why feature selection was done first).

7 Experimental Setup

Python Version	3.13
Scikit-Learn Version	1.5.x
IDE	Jupyter Notebook (VS Code)
Operating System	Windows 11
Hardware	Intel i5, 16GB RAM

8 Hyperparameter Selection

Parameter	Value
Feature Selection Method	SelectKBest (ANOVA F-test)
Number of features selected (k)	5
Test size	0.2
Validation size	0.1
Random state	42
Scaling	StandardScaler

Since plain Linear Regression does not really have hyperparameters to tune (its a closed form solution), the only real "hyperparameters" for this experiment were the ones controlling the pipeline itself, mainly how many features to keep (k=5) and the train/-val/test ratios.

9 Results

Metric	Value
MAE	0.379
MSE	0.275
RMSE	0.524
R ² Score	0.686

Note: Since this is a regression problem the classification metrics (Accuracy, Precision, Recall, F1, ROC-AUC) from the template do not really apply here, MAE/MSE/RMSE/R2 were used instead which is what the `evaluate_model()` function returns for `task="regression"`.

Listing 4: Console output – Model evaluation

```

1 FEATURE SELECTION
2 Selected Features:
3 ['AccelSec', 'FastCharge_KmH', 'RapidCharge', 'PlugType', '
   Segment']
4 DATA SPLITTING
5 Training Shape : (71, 5)
6 Validation Shape : (11, 5)
7 Testing Shape : (21, 5)
8
9 REGRESSION MODEL
10 MODEL EVALUATION
11 MAE : 0.3788909542438544
12 MSE : 0.27482686024636516
13 RMSE : 0.5242393158151772
14 R2 Score : 0.6860123848598272

```


10 Result Visualization

The correlation heatmap and outlier/feature distribution plots generated by `perform_eda()` (Figures 1 and 2 above) double up as the main visualizations for this experiment, since a full pipeline covering EDA + preprocessing + modelling was the actual goal here rather than just one algorithm. A dedicated confusion matrix / ROC curve is not applicable in this run because the task turned out to be regression and not classification, note that the `evaluate_model()` function does plot a confusion matrix automatically whenever `task="classification"` is passed, that part will be used in the next experiment when we work with the Iris or Diabetes datasets.

- **Correlation Matrix:** Shown in Figure 1, subplot 2. Helps identify which features are actually related to price before doing feature selection.
- **Feature Means / Distribution:** Shown in Figure 1 and 2, subplots 3, 4, 8, 9, 10. Useful to see how scaling changed the spread of each feature.
- **Outlier Count:** Subplot 11 in both figures, useful to check whether IQR clipping actually reduced the number of extreme values in each column.

11 Discussion

This experiment was less about "does the model perform well" and more about getting the workflow right, so the R^2 of 0.686 is honestly a decent starting point for a linear model on a small 103-row dataset, but its definitely not great and there is scope for improvement. A few things that could be tried next time: use a Random Forest or Gradient Boosting regressor instead of plain Linear Regression since the relation between EV specs and price is probably not fully linear, try a different k value for feature selection (5 might be too few, some information could be getting dropped), and maybe use RobustScaler instead of StandardScaler given how many outliers were present in the raw price column.

One thing I noticed while running this is that `SelectKBest` with the ANOVA test threw a warning about constant features and division by zero – this happened because after Label Encoding some categorical columns basically became near constant after clipping, so this is something to keep in mind when combining outlier clipping with encoding + feature selection in that exact order, the order of operations in the pipeline actually does matter here.

Overall the modular design (seperate functions for EDA, preprocessing, feature selection, splitting and evaluation) worked out well, it made testing and debugging each part individually a lot easier, and this same set of functions can basically just be re-used as-is for the classification/clustering experiments coming up later in the semester.

12 Conclusion

In this experiment, the Numpy, Pandas, Scipy, Scikit-learn and Matplotlib libraries were explored practically by building a small end-to-end, reusable ML pipeline covering EDA, preprocessing, feature selection, data splitting and model evaluation, and testing it on the Electric Car Data dataset for a price prediction (regression) task. The Linear Regression model gave a reasonable R^2 score of 0.686 on the test set. Along with the coding part,

five real world datasets (Iris, Loan Amount, Diabetes, Spam, MNIST) were studied and classified according to the type of ML task they represent, which gave a good overview of how different problems (classification vs regression, tabular vs text vs image data) need different preprocessing and feature selection strategies. This foundation should make it a lot easier to jump straight into the actual algorithm-focused experiments from here on.

13 References

- [1] Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] W. McKinney, “Data Structures for Statistical Computing in Python,” *Proceedings of the 9th Python in Science Conference*, pp. 56–61, 2010.
- [3] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [4] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [5] Kaggle, “Electric Vehicle Dataset,” [Online]. Available: <https://www.kaggle.com/datasets/geoffnel/evs-one-electric-vehicle-dataset>.
- [6] UCI Machine Learning Repository, [Online]. Available: <https://archive.ics.uci.edu/>.